



2025年度中国青年

揭榜挂帅擂台赛 · 华为赛道



目录



1 昇腾CANN

2 昇腾芯片

3 昇腾编程模式

4 昇腾并行模式

5 AscendC

6 AscendC算子优化

7 答疑环节



昇腾CANN (Compute Architecture for Neural Networks)



AI框架

全面支持业界AI框架，适配PyTorch社区版本



CANN

Ascend C 算子编程语言

支持算子极简开发
高底层API | 算子模板库

算子加速库

覆盖Transformer网络
提供高性能融合算子/基础算子

HCCL 集合通信库

高效实现分布式通信
高性能集合通信算法

GE 图引擎

图编译、图执行，使能处理
器计算加速，提升模型性能

MindStudio 全流程工具链

算子开发工具
训练开发工具
推理开发工具
可视化工具

BiSheng Compiler 毕昇编译器

异构编译 | 微架构优化 | 指令亲和

Runtime 运行时

控制流 | 内存管理 | 任务调度 | 设备管理

Driver 驱动

昇腾硬件

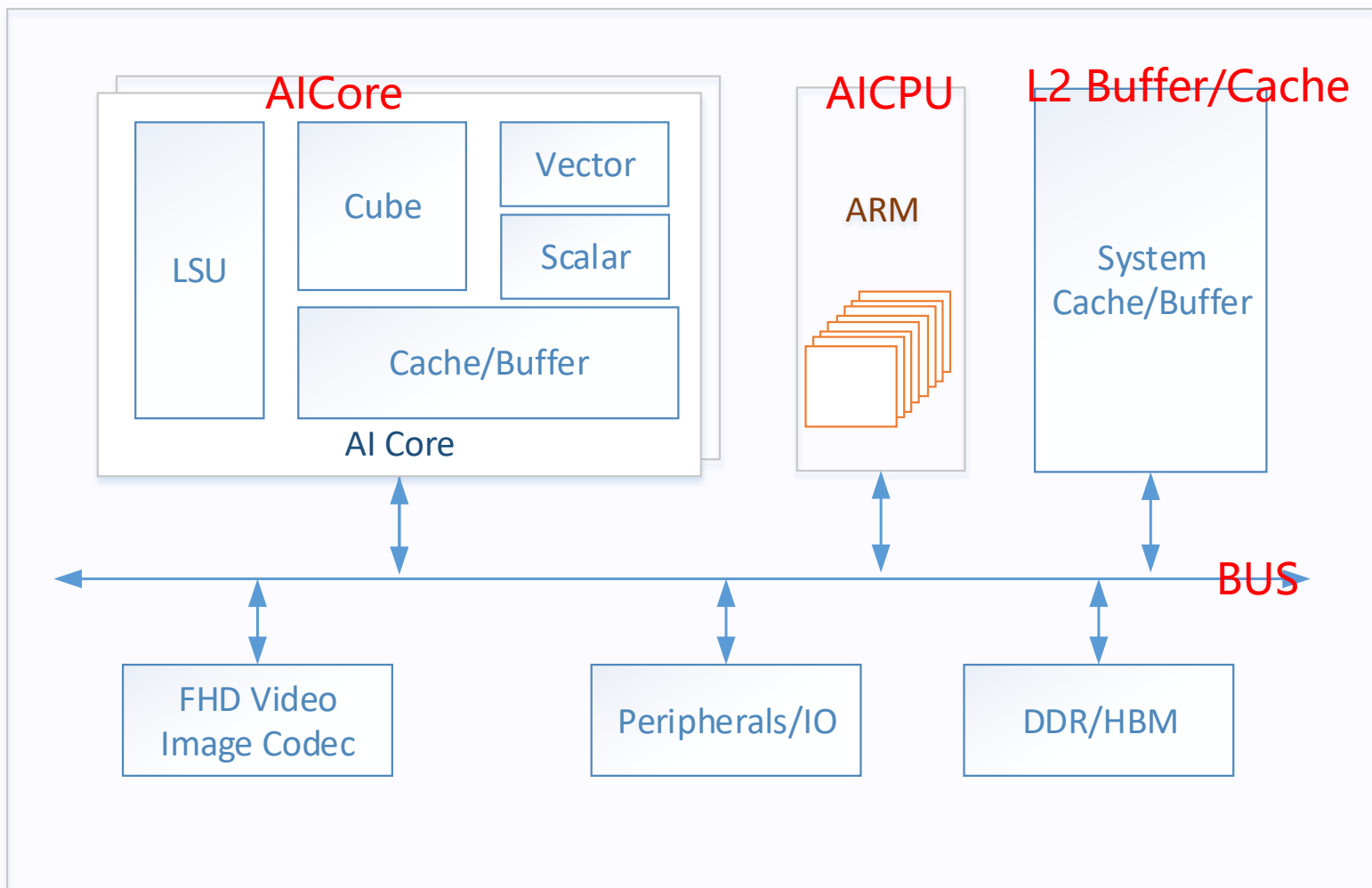


昇腾系列处理器





昇腾310/910 芯片结构示意图

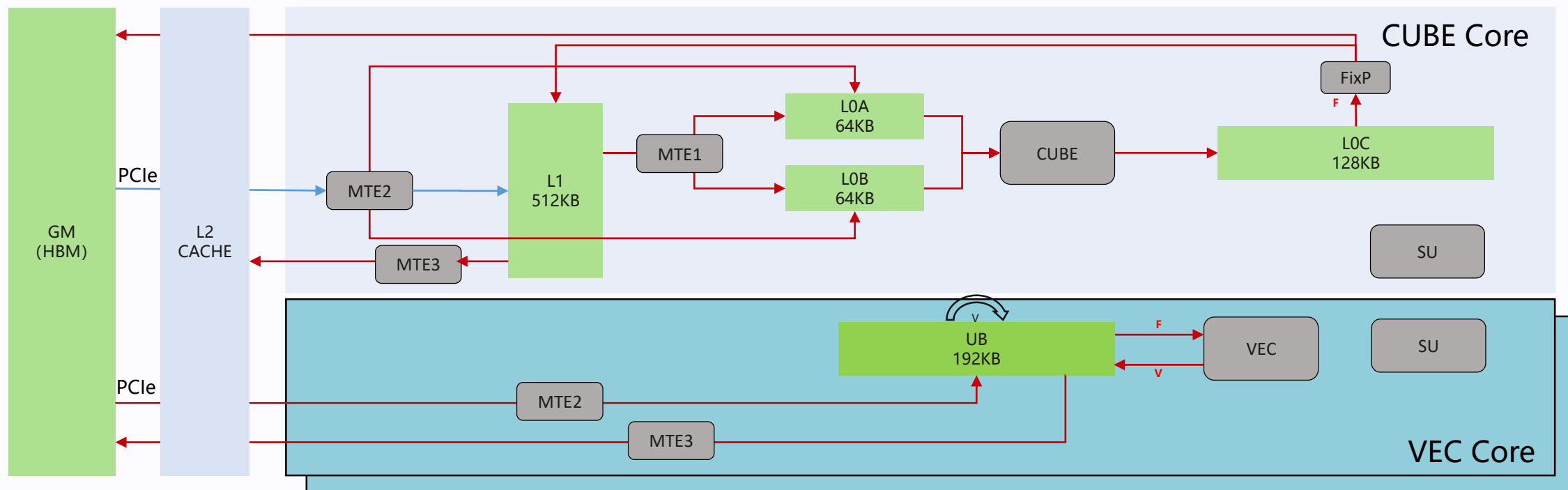


➤ L2 Buffer vs L2 Cache

- 同一个介质，两种使用模式
- Buffer: 程序员可见并可以直接读写（地址空间和DDR/HBM不重合）
- Cache: 作为DDR/HBM高速缓存，程序员不可见

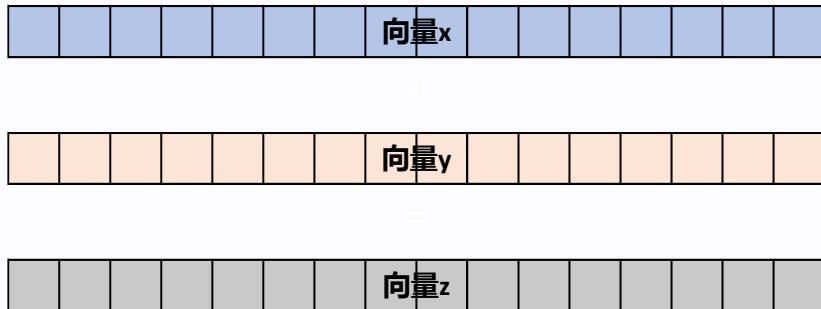
➤ DDR/HBM

- DDR: 普通内存，带宽低/价格低
- HBM: High Bandwidth Memory, 带宽高/成本高



- 910B CUBE核架构, :
 - 5个异步并行执行单元: MTE1, MTE2, MTE3, CUB, **SU, (还有1个FixP(FixPipe))**
 - 多个内存单元: GM, L1, L0A, L0B, L0C, (**L2开发者不可控制, 所有GM的内存操作都要经过L2 CACHE**)
- VEC核架构:
 - 4个并行执行单元: MTE2, MTE3, VEC, SU
 - 2个内存单元: GM, UB
- 典型计算数据流
 - VEC计算:
 - GM-(MTE2)-UB- (VEC)-UB-(MTE3)-GM;
 - CUB计算:
 - GM-(MTE2)-L1-(MTE1)-L0A/L0B-(CUBE)-L0C-(FixP)-GM

昇腾编程模式：标量、向量和矩阵计算



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1															
2															
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															

矩阵A

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1															
2															
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															

矩阵B

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1															
2															
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															

矩阵C

标量计算
int x=0, y=0;
int z=x + y;

向量计算, NPU一个核可以在1个时钟周期处理128个FP16的加法
int x[1024], y[1024], z[1024];
...
Add(z, x, y, 1024); //z=x + y;

矩阵计算 $C=A * B$,
NPU一个核可以在1个时钟周期处理完成 $16 * 16 * 16$ 的矩阵乘法FP16数据类型
的矩阵乘
Mmad(C, A, B, M, K, N);

SIMD, 也就是单指令多数据计算, 一条指令可以处理多个数据:
Ascend C编程API主要是向量计算API和矩阵运算API, 计算API都是SIMD样式。

昇腾并行模式：SPMD 和流水线并行



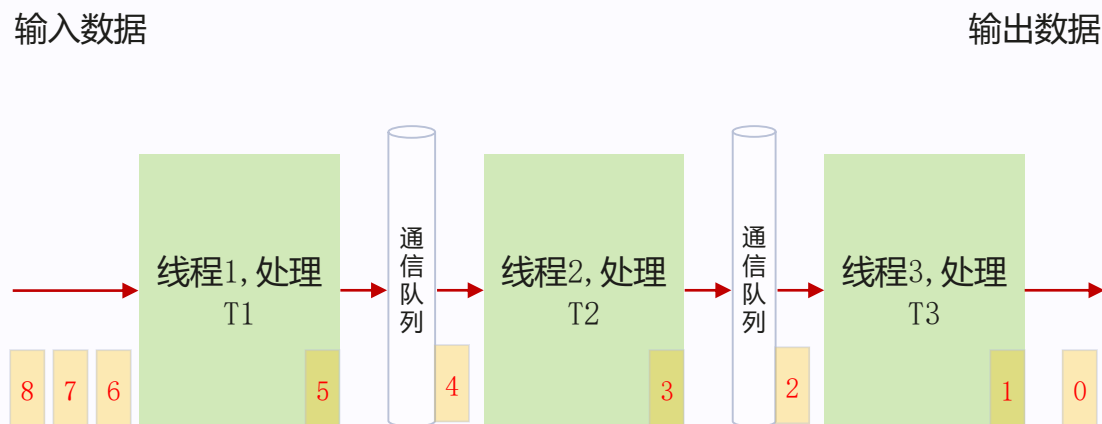
SPMD并行



SPMD 数据并行计算原理

- 启动一组线程实例，他们运行的相同程序
- 把待处理数据切分，把切分后数据分片分发给不同线程处理
- 每个线程对自己的数据分片进行3个任务T1、T2、T3的处理

流水并行



流水线并行原理

- 启动一组线程
- 对数据进行切分
- 每个线程都处理所有的数据切片，对输入数据分片只做一个任务的处理



AscendC介绍



Ascend C算子采用标准C++语法和一组类库API进行编程

- Ascend C类库基本API
 - 计算API: 向量计算API, 矩阵计算API
 - 搬运API: 数据搬运API
 - 同步API: 队列操作 API, 内存管理 API
- Ascend C类库API是类SIMD方式, 计算操作数都是Tensor类型: **GlobalTensor**和**LocalTensor**



向量计算API举例

```
dstLocal = src0Local + src1Local;
Add(dstLocal, src0Local, src1Local, 512);
```

矩阵计算API举例

```
// 左矩阵形状为[m, k], 右矩阵形状为[k, n], 计算结果搬出至GM, 目的矩阵
// 无需初始化
Gemm(dstC01, featureMapA1, weightB1, m, k, n, tilling, false,
false);
```

数据搬运API举例

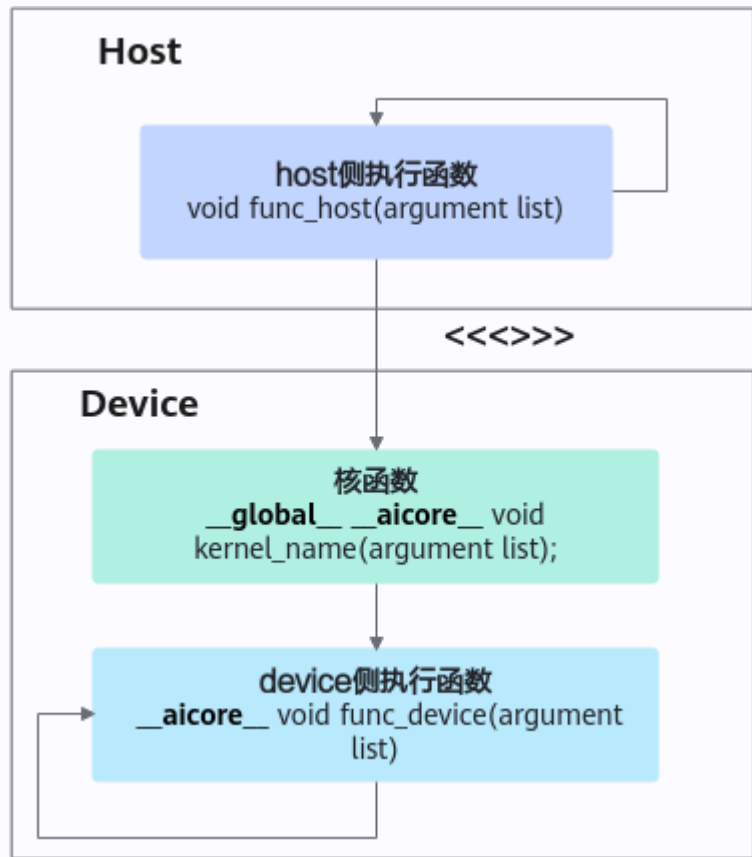
```
DataCopy(src0Local, src0Global, 512);
DataCopy(src1Local, src1Global, 512);
```

队列和内存操作API举例

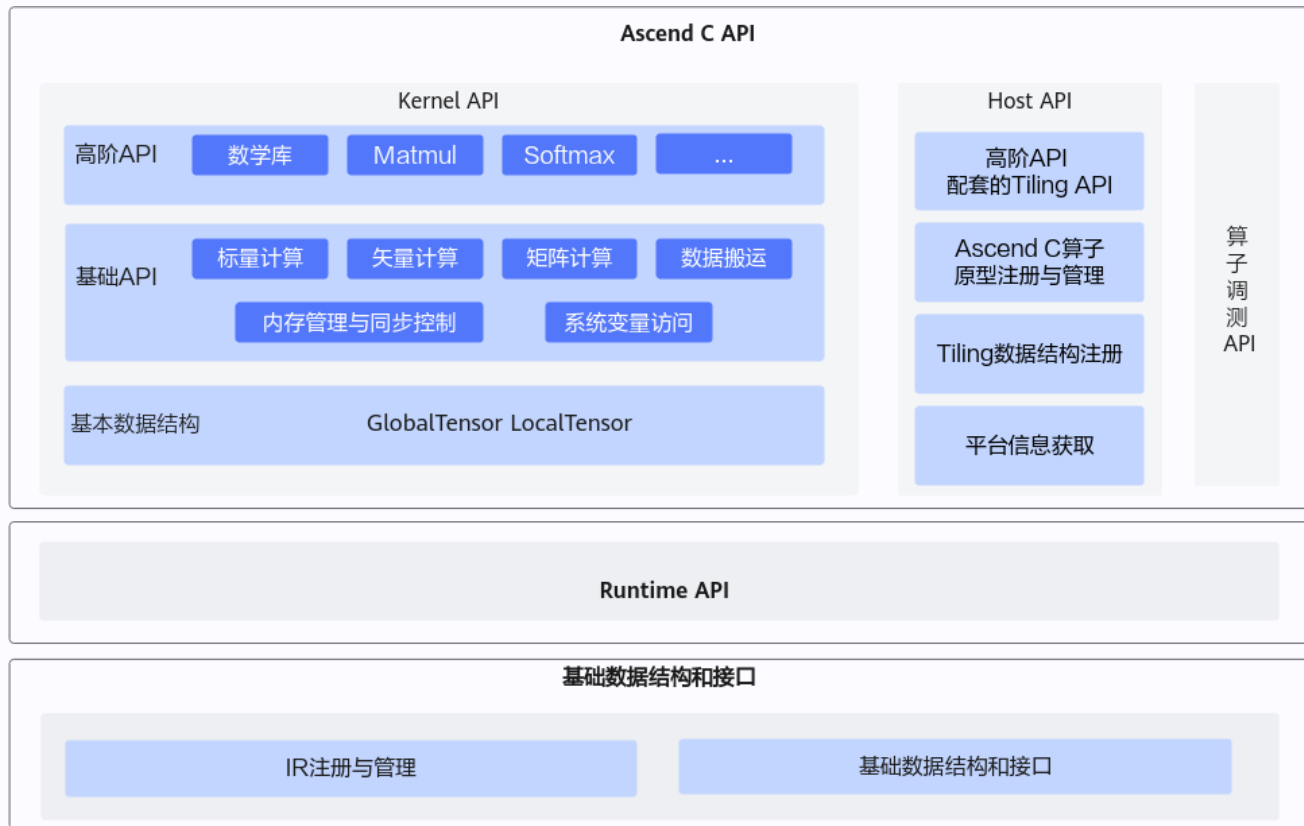
```
que.Enqueue(tensor1);
LocalTensor<half> tensor2 = que.DeQueue<half>(); // 将tensor从VECOUT的Queue中取出
LocalTensor<half> tensor1 = que.AllocTensor();
que.FreeTensor<half>(tensor1);
```



程序结构



编程API



样例 <https://gitee.com/ascend/samples>



Ascend C支持**SPMD数据并行**，将需要处理的数据拆分并分布在多个计算核心上运行，多个AI Core共享相同的指令代码

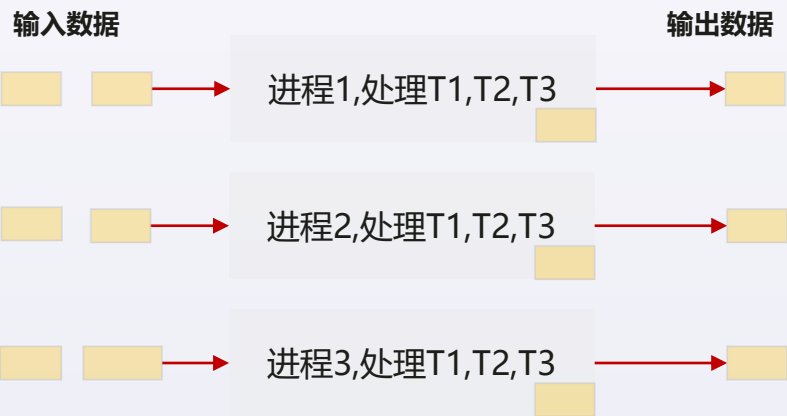
SPMD (Single Program Multiple Data) 单程序多数据并行模式

单程序

所有任务都执行同一个程序的拷贝

多数据

不同的任务操作不同的数据



启动一组程序，他们运行相同的程序

把待处理数据切分，把切分后的数据分发给不同进程处理

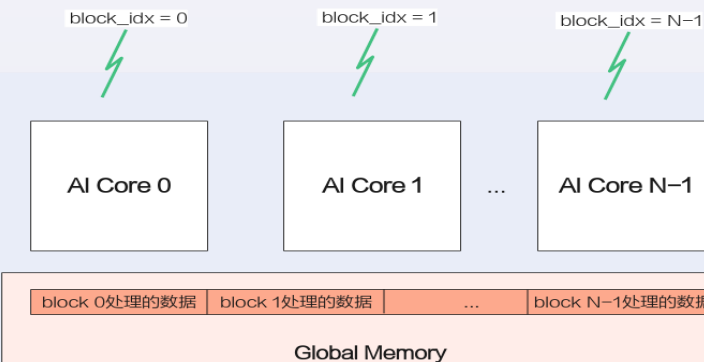
每个进程对自己的数据分片进行3个任务（T1、T2、T3）的处理

开发者仅需关注单核算子实现

每个核上的运行实例唯一的区别是block_idx不同

```
18 class KernelAdd {
19 public:
20     __aicore__ inline KernelAdd() {}
21     __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)
22     {
23         // 获得本block要计算的数据位置
24         xGm.SetGlobalBuffer((__gm__ half*)x + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
25         yGm.SetGlobalBuffer((__gm__ half*)y + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
26         zGm.SetGlobalBuffer((__gm__ half*)z + BLOCK_LENGTH * GetBlockIdx(), BLOCK_LENGTH);
27         // 初始化对列的内存资源
28         pipe.InitBuffer(inQueueX, BUFFER_NUM, TILE_LENGTH * sizeof(half));
29         pipe.InitBuffer(inQueueY, BUFFER_NUM, TILE_LENGTH * sizeof(half));
30         pipe.InitBuffer(outQueueZ, BUFFER_NUM, TILE_LENGTH * sizeof(half));
31     }
32     ...
33 }
```

每个block用内嵌变量block_idx来识别自己的身份





AscendC介绍



提供底层API，高层API，支持创新算子开发

底层API 80+ → 130+

实现对硬件能力的抽象，开放芯片的能力，保证完备性和兼容性。

新增底层API 50+

标量计算	ScalarGetCountOfValue、 ScalarCountLeadingZero	↑ 5
矢量计算	AddRelu、AddReluCast、 BilinearInterpolation.....	↑ 8
矩阵计算	LoadData、Mmad、Conv2D、Gemm	
数据搬运	DataCopy、DataCopyPad、Copy	↑ 1
内存管理与同步控制	SetFlag/WaitFlag、PipeBarrier、InitBuffer	↑ 15
系统变量访问	GetArchVersion、GetSystemCycle、 GetSubBlockNum.....	↑ 8
调测接口	DumpTensor、printf、Trap	↑ 6
其他	ToBfloat16、ToFloat、SetAtomicType、Gather...	↑ 16

高层API 10+ → 70+

实现一些常用的计算算法，用于提高编程开发效率，通常会调用多种底层API实现。

高层API包括数学库、Matmul、Softmax等API。

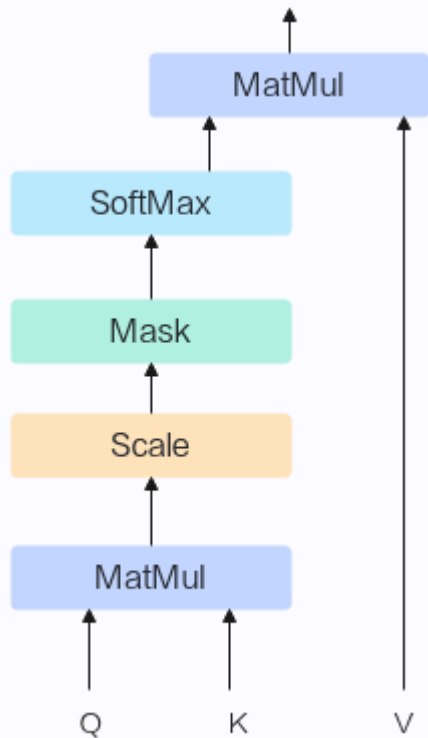
新增高层API 60+

数学运算	Tanh、Asin、Sin、Acos、Cos、Log、Atan、Power、Sinh
矩阵乘	Matmul
激活函数	SoftMax、LogSoftMax、Gelu、FasterGelu、SwiGLU、 Sigmoid
数据归一化	RmsNorm、BatchNorm、DeepNorm
量化反量化	AscendQuant、AscendDequant、AscendAntiQuant
规约操作	Sum、Mean
排序	Topk
其他	DropOut、ArithProgression、ConfusionTranspose

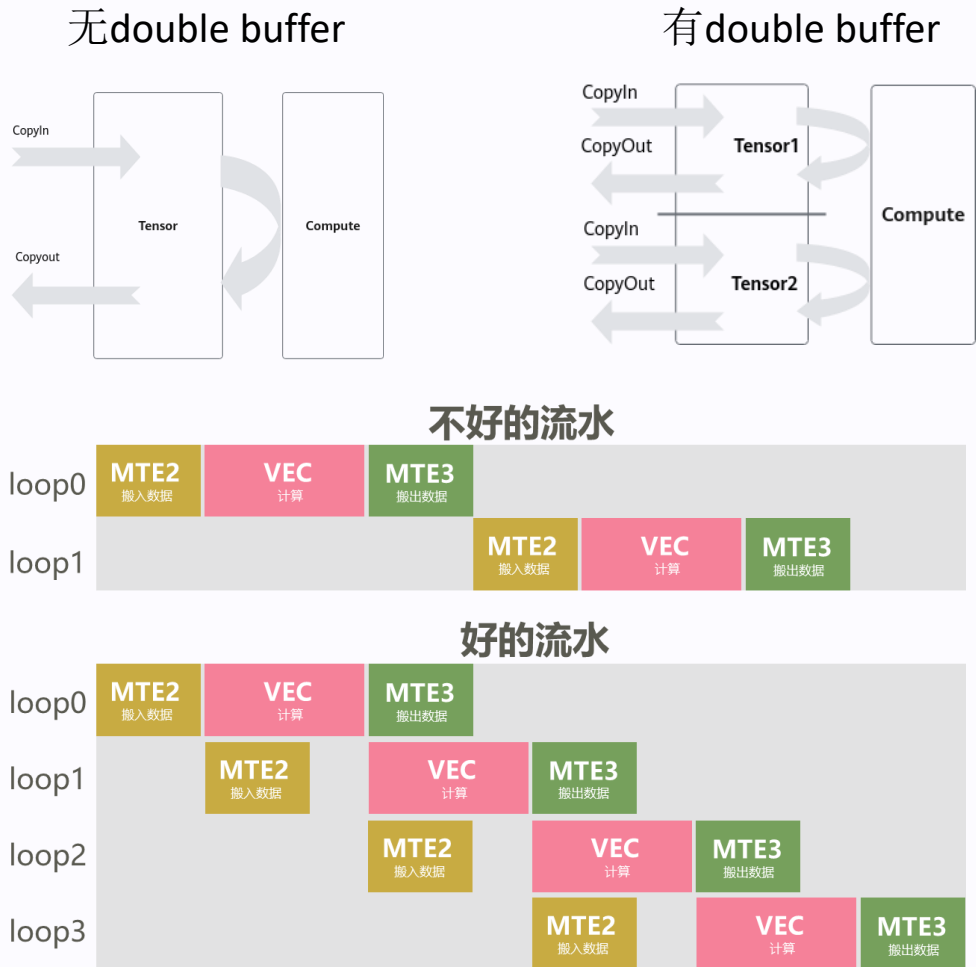


融合算子

融合算子由多个独立的小算子融合而成，其功能与多个小算子的功能等价，性能方面通常优于独立的小算子。用户可以根据实际业务场景诉求，按照具体算法自由融合向量（**Vector**）、矩阵（**Cube**）算子以达到性能上的收益。



流水优化



Ascend C最佳实践

前言

异构计算

功能调试

性能分析

性能优化

搬运优化

内存优化

API使用优化

流水优化

Tiling优化

优秀实践

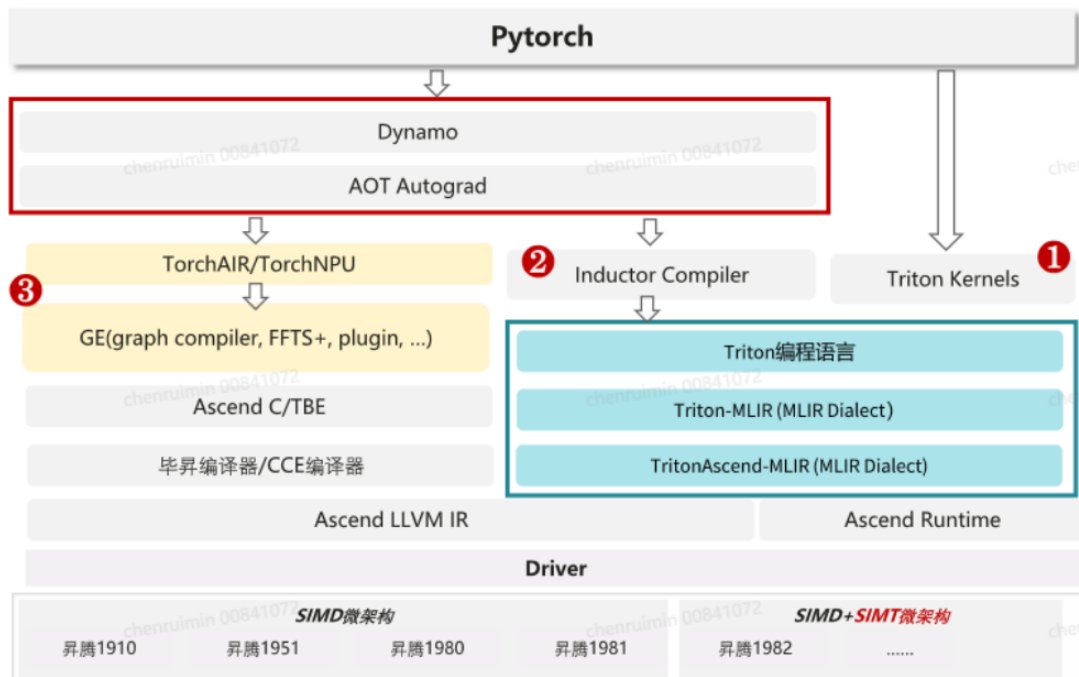


TorchAir（Torch Ascend Intermediate Representation）是Ascend Extension for PyTorch（torch_npu）中支持图模式能力的扩展库，支持PyTorch网络在昇腾设备上进行图模式推理。

TorchAir对接PyTorch的Dynamo特性，将PyTorch的FX（Functionalization）图转换为Ascend IR（Intermediate Representation）计算图，通过GE进行计算图的编译优化等操作，并下发到昇腾硬件执行

入图模式

```
import torch_npu, torchair
config = torchair.CompilerConfig()
# 设置图下沉执行模式
config.mode = "reduce-overhead"
npu_backend = torchair.get_npu_backend(compiler_config=config)
opt_model = torch.compile(model, backend=npu_backend)
```



参数说明

设置图下沉执行模式，字符串类型。

- max-autotune（缺省值）：表示Ascend IR Graph下沉模式，其具备了一定的图融合和下沉执行能力，但要求所有算子都注册到Ascend IR计算图，
- reduce-overhead：表示aclgraph下沉模式，主要是单算子kernel的下沉执行，暂不具备算子融合能力，不需要算子注册到Ascend IR计算图。当PyTorch模型存在Host侧调度问题时，建议开启此模式。



昇腾开发工具



命令行工具集

训练开发

分析迁移

精度调试

性能调优

推理开发

模型转换

模型压缩

精度调试

性能调优

算子开发

算子设计 | 异常检测 | 算子调优 | 算子调试

算子工程

可视化工具

MindStudio Insight



算子评判标准



1. 算子功能要求

与Golden结果精度误差高于0.01%将不得分

2. 定义算子类型难度

- (1) 是不是vector算子（较低难度）
- (2) 是不是cube算子（中等难度）
- (3) 混合算子（较高难度）：既有vector，也有cube算子

3. 每个难度再按照层级给分

层级1：性能提升幅度（（华为 CANN 或同算力 GPU 的等效实现）相比，计算性能提升越大，得分越高。）

层级2：创新性加分：如果是新算子，将获得额外加分

层级3：支持的输入/输出 Tensor 的 shape 覆盖范围更广（能处理更多样化、更极端的维度组合，如超大/超小尺寸、非规则形状、高维张量），得分越高。

算子名称	应用场景	支持数据类型	支持芯片	优化的评判标准	预期目标/价值
SageAttention	LLM模型、文生图模型、视频生成模型	输入：FP16 输出：FP16	910B、910C	性能提升2+倍，模型端到端精度损失<1%	实现低精度模型量化、加速attention计算



答疑环节

选手自行提问

THANK YOU

